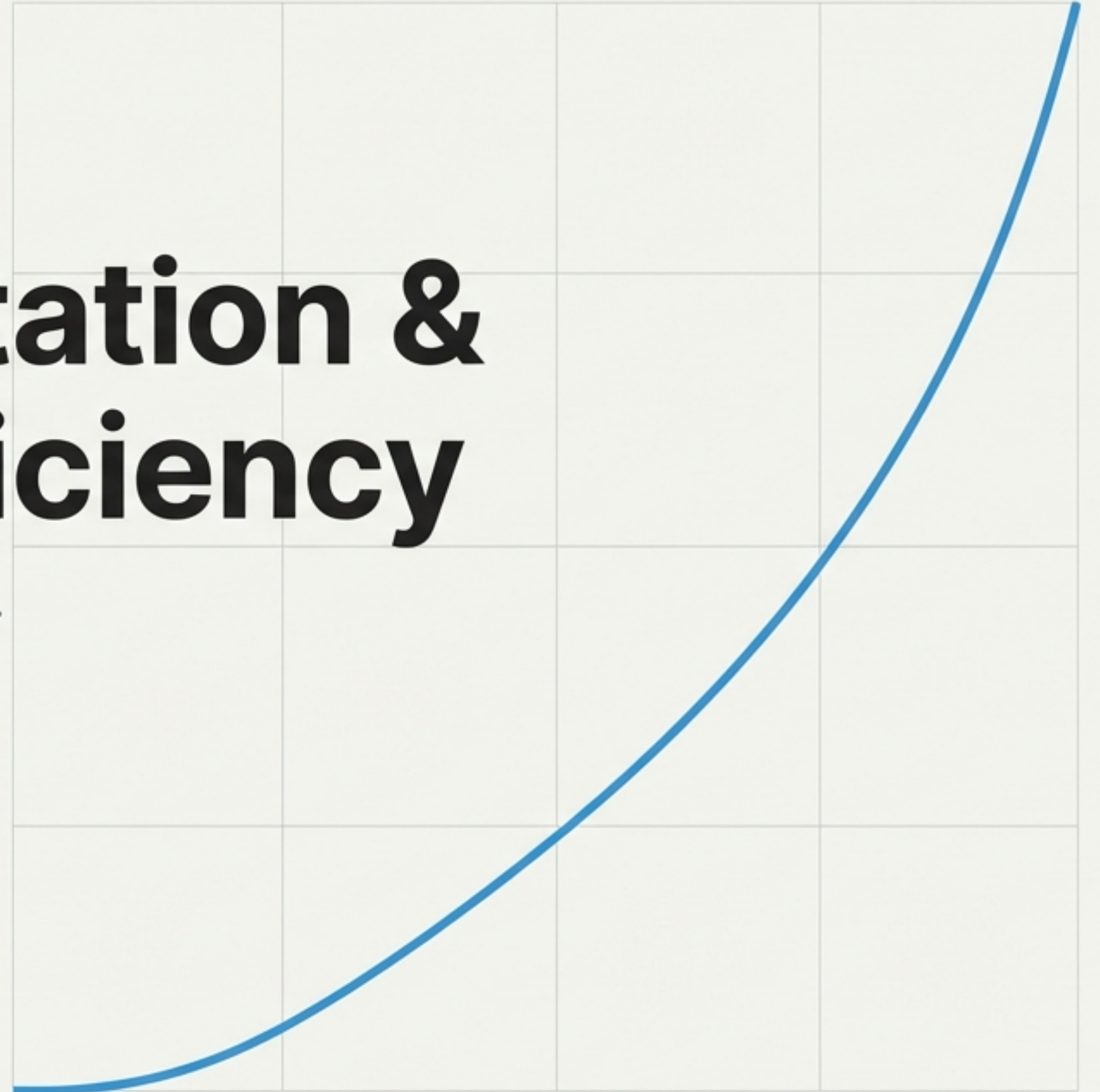


Asymptotic Notation & Algorithmic Efficiency

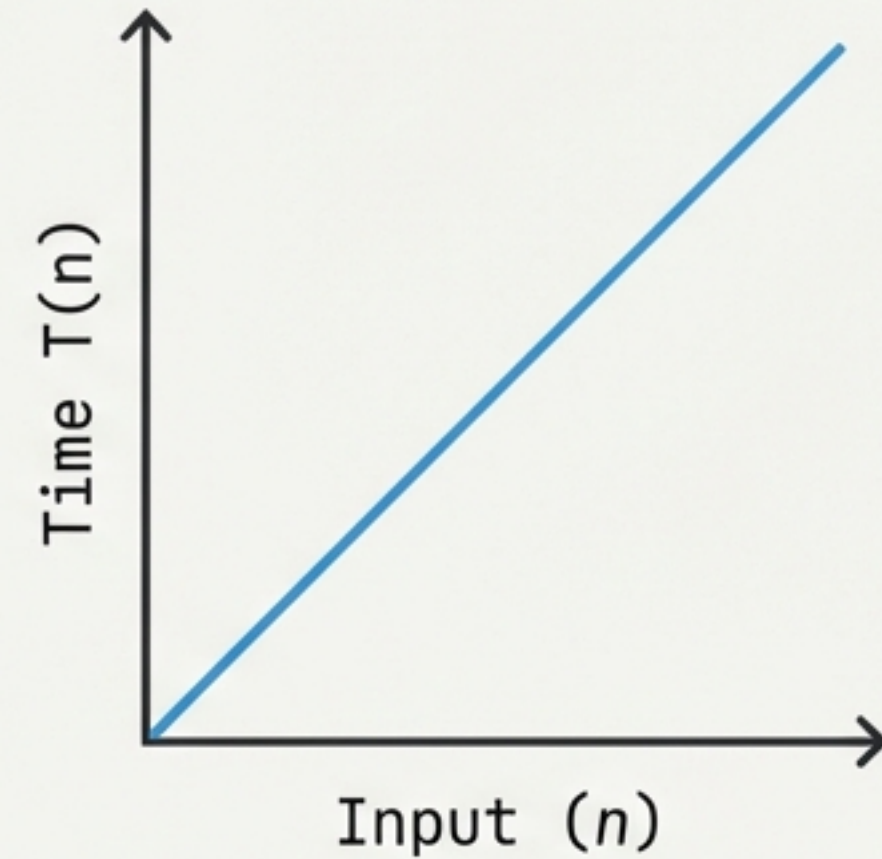
A framework for predicting system behavior and measuring performance at scale.



We Measure Growth, Not Seconds



Measurement: Seconds
Dependency: Machine Hardware
Status: Irrelevant



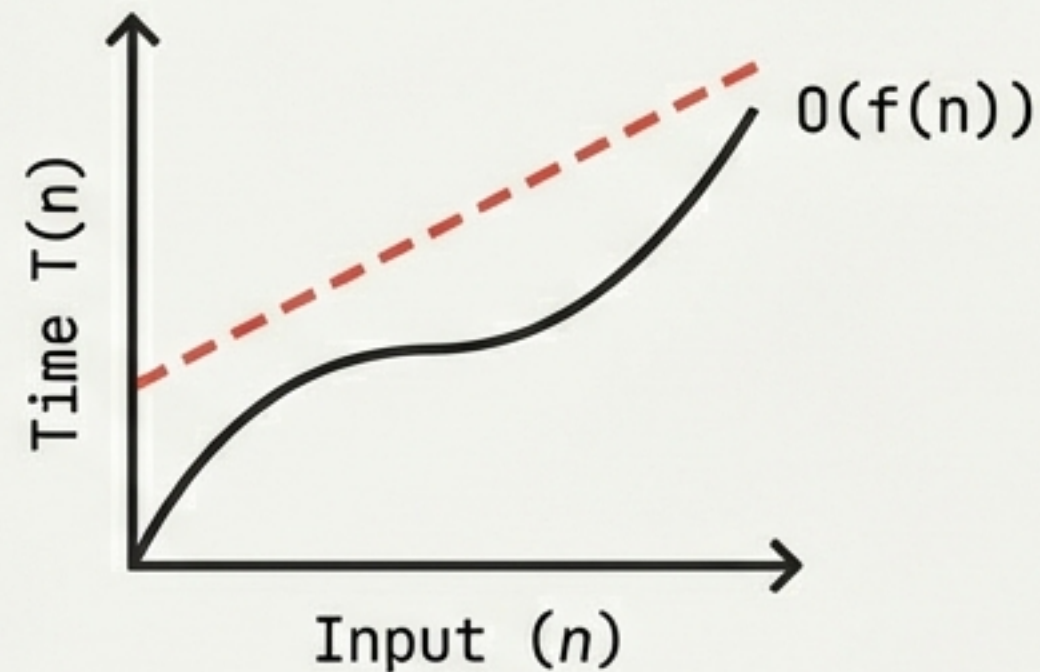
Measurement: Growth Rate
Dependency: Algorithm Logic
Status: Critical

Asymptotic Analysis: Measuring how time $T(n)$ grows as input size n grows.

The Three Bounds of Performance

Big O (O)

The Upper Bound

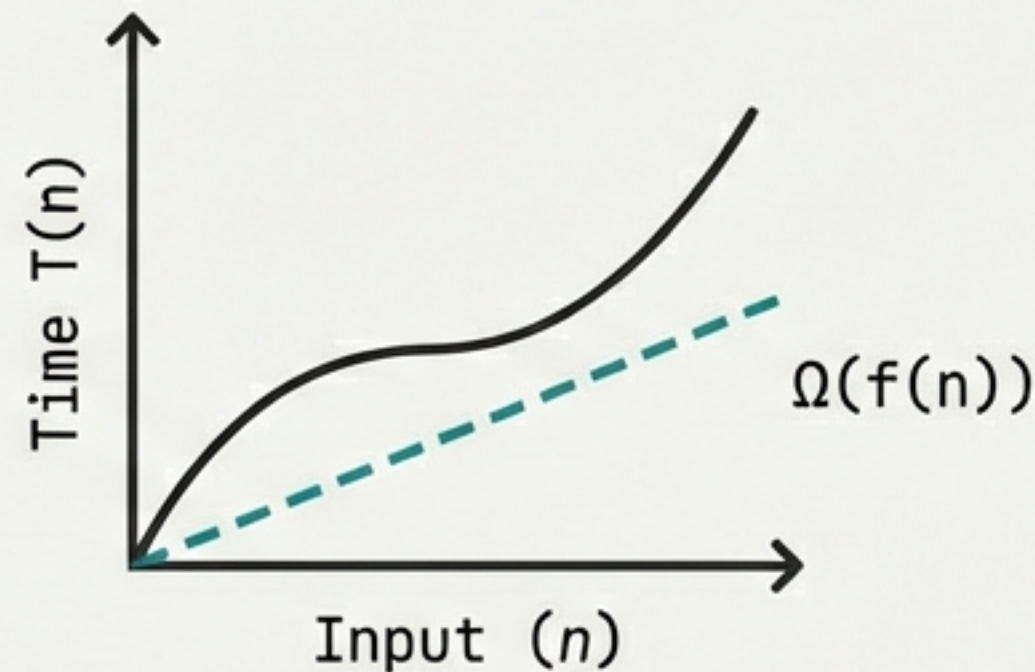


Meaning: Maximum time / Worst Case.

Analyst Note: It will never be slower than this.

Omega (Ω)

The Lower Bound

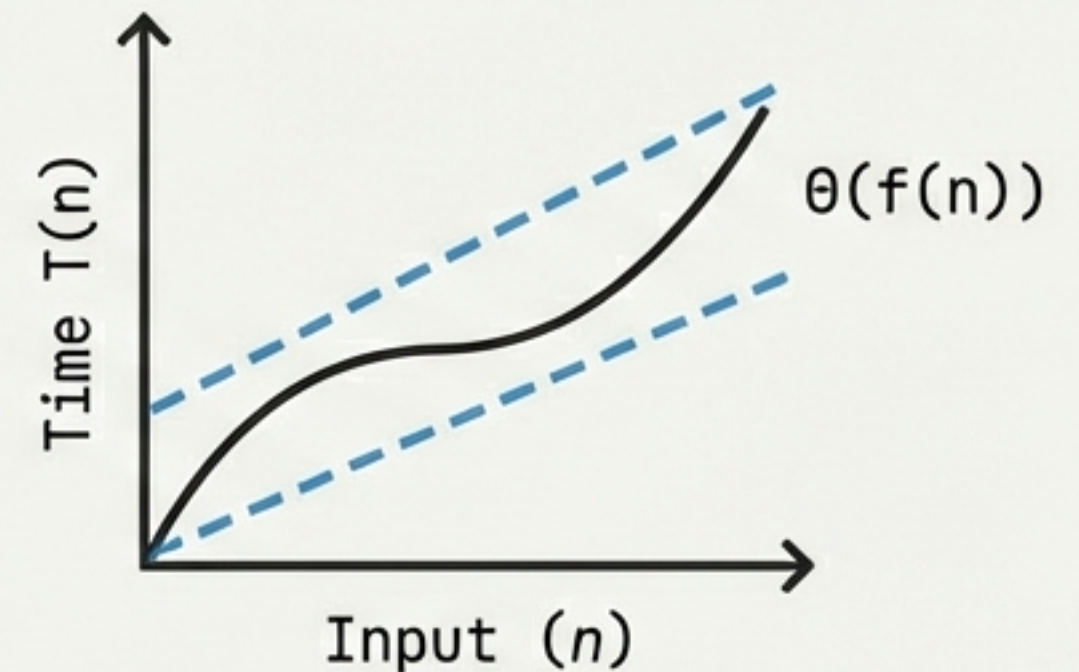


Meaning: Minimum time / Best Case.

Analyst Note: It will never be faster than this.

Theta (Θ)

The Tight Bound



Meaning: Exact growth rate.

Analyst Note: It will always be roughly this.

Case Study I: Input Configuration Dictates Performance

The Scenario

Searching for a target number in an array.



```
for i in range(n):  
    if arr[i] == target:  
        return i
```

Best Case (Ω)

Target is at the first position [Index 0].
Found in 1 step.

$$\Omega(1)$$

Worst Case (O)

Target is at the last position or not present.
Check all n elements.

$$O(n)$$

Key Insight: Because the minimum time (1) and maximum time (n) differ, we cannot assign a Theta (Θ) value.

Case Study II: The Certainty of Tight Bounds

The Scenario

Printing every pair of numbers (i, j).

```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```

The Analysis

Total Operations = $n \times n$											

Minimum Time: n^2 steps
Maximum Time: n^2 steps

Tight Bound: $\Theta(n^2)$

The Hierarchy of Algorithmic Complexity

Efficient

$O(1)$ - Constant Time

Time does not change with n (e.g., Array Access)

$O(\log n)$ - Logarithmic

Cuts problem in half each step (e.g., Binary Search)

$O(n)$ - Linear

Time increases directly with input (e.g., Simple Loop)

Manageable

$O(n \log n)$ - Linearithmic

Efficient sorting (e.g., Merge Sort)

Expensive to
Impossible

$O(n^2)$ - Quadratic

Nested loops (e.g., Bubble Sort)

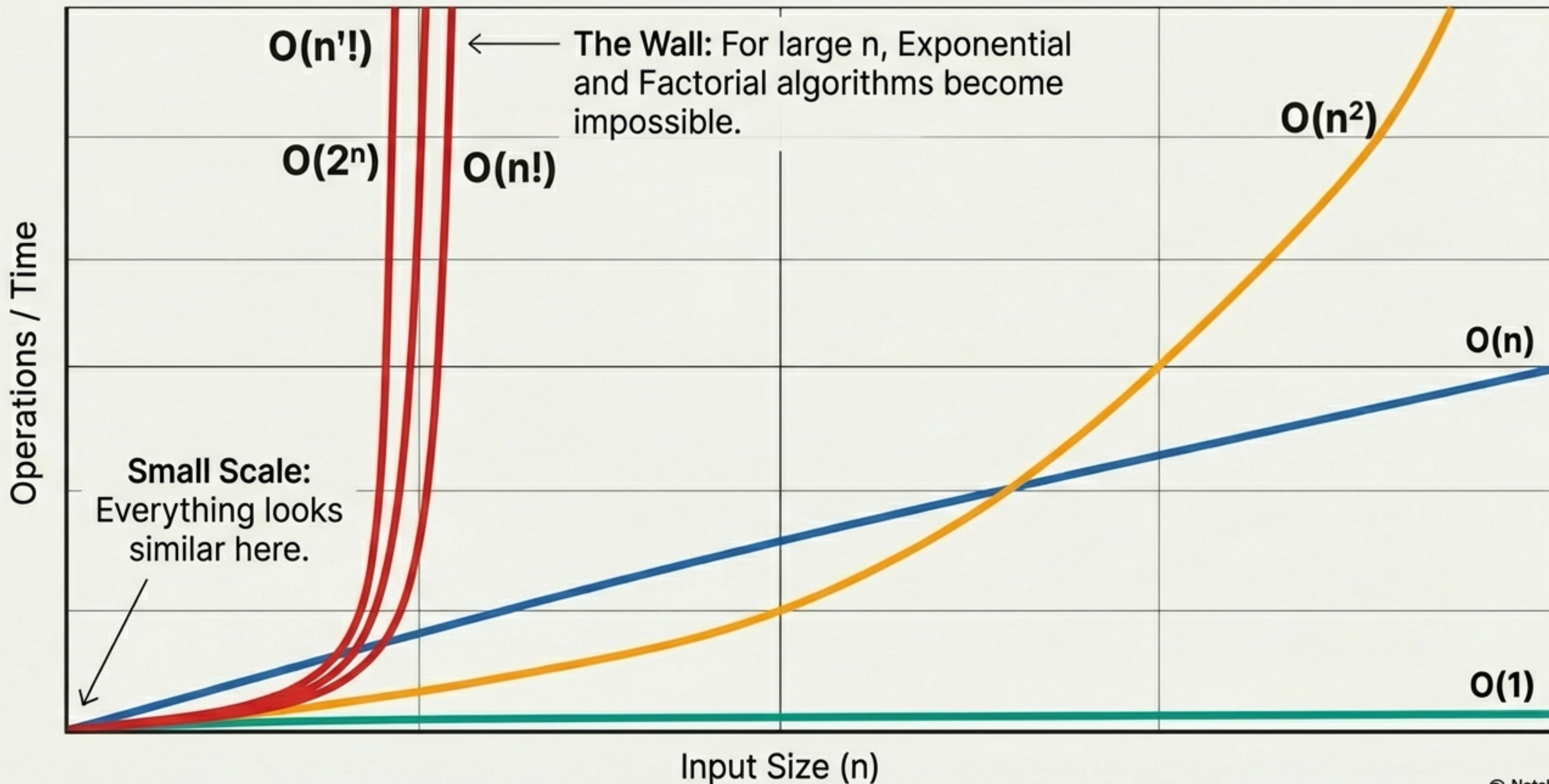
$O(2^n)$ - Exponential

Doubles every step (e.g., Naive Recursion)

$O(n!)$ - Factorial

All permutations (e.g., Traveling Salesman)

Visualizing the Growth Explosion



The Scale of Difference (n=10)

Raw operation counts for a small input size of 10.

Complexity Class	Growth Type	Operations Count
$O(n)$ (JetBrains Mono)	Linear	10
$O(n^2)$ (JetBrains Mono)	Quadratic	100
$O(2^n)$ (JetBrains Mono)	Exponential	1,024
$O(n!)$ (JetBrains Mono)	Factorial	3,628,800

Takeaway: Even with a tiny input of 10 items, an $O(n!)$ algorithm performs over 3.6 million operations.

The Rules of Simplification

As n approaches infinity, we focus only on the dominant term.

Rule 1: Ignore Constants

$$\cancel{100}n + \cancel{5} \longrightarrow O(n)$$

For large n , the factor of 100 becomes negligible compared to the growth trend.

Rule 2: Drop Lower Order Terms

$$n^2 + \cancel{n} + \cancel{5} \longrightarrow O(n^2)$$

The n^2 term dominates the result so heavily that smaller terms are irrelevant.

Heuristics for Pattern Recognition

Quick visual cues in code structure to estimate time complexity.



JetBrains Mono

```
for i in range(n)
```

Deep Charcoal

$O(n)$



JetBrains Mono

```
for i... for j...
```

Critical Red

$O(n^2)$



JetBrains Mono

Binary Search / Halving

Efficiency Teal

$O(\log n)$



JetBrains Mono

```
for i... for j in range(i)
```

Critical Red

$O(n^2)$

Inter

Still Quadratic (avg $n^2/2$)



JetBrains Mono

```
for i... for j in range(5)
```

Deep Charcoal

$O(n)$

Inter

Inner loop is constant.

Summary & Key Takeaways



Big O (O): Maximum Time (Safety)



Omega (Ω): Minimum Time (Optimism)



Theta (Θ): Exact Growth (Precision)

Purpose: We study asymptotic notation to compare algorithms effectively for large inputs, ensuring our systems scale gracefully.

The Golden Rule: Strive for $O(1)$ or $O(\log n)$. Avoid $O(n!)$ at all costs.

Efficiency



Thank You.